Elements of Software Construction | Final Presentation

By Cohort 3 Team 2

Scan. Grow. Battle.

Sprout

Web Platform & B2B Showcase for Nature-Based Gamification

Class of 2028

Nathaniel Sim, Justin Teh, Omar,
Andrina, Li Xiang, Zhi Feng

Do you remember when you last visited a garden?





High foot traffic,
low lasting engagement





- Little meaningful engagement
- **Apathy** towards the environment

problems to explore

problem statement

how might we shift high foot traffic into **lasting environmental awareness**, rather than just Instagram-able moments?

a gamified and sticky experience for tourists to learn about plants

Sprout



Sprout’s Requirement

Recap Summary

Full stack web app

- Database shared with mobile app
- All notifications sent by email (no ‘SMS’)

Internal components are not actors

- Database
- Gameplay Mechanism
- Image preprocessing, etc

Not Implemented

- PVP, B2B advertisement and B2B dashboard

Sprout’s Requirement

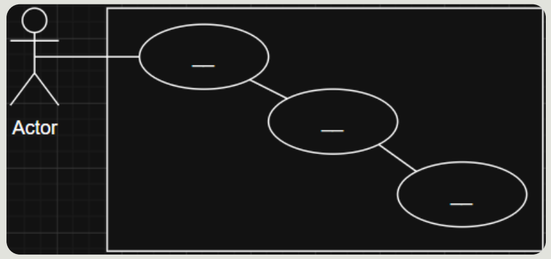
General Rules from notes & feedback

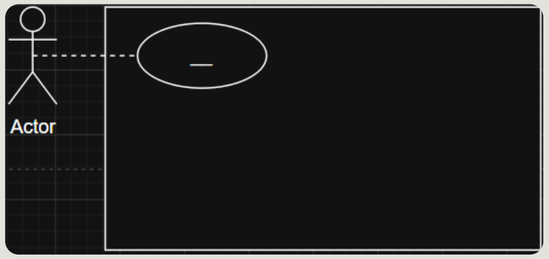
Documentation

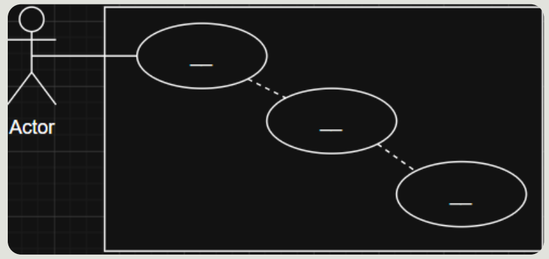
- Actor (Defined by session state)
- Proper naming convention
- Preconditions weakened to NIL (when applicable)
- Common Requirements (Shared preconditions)
- Flows always starts with the actor

Avoids

- base to base
- actor to sub
- sub to base







Actors

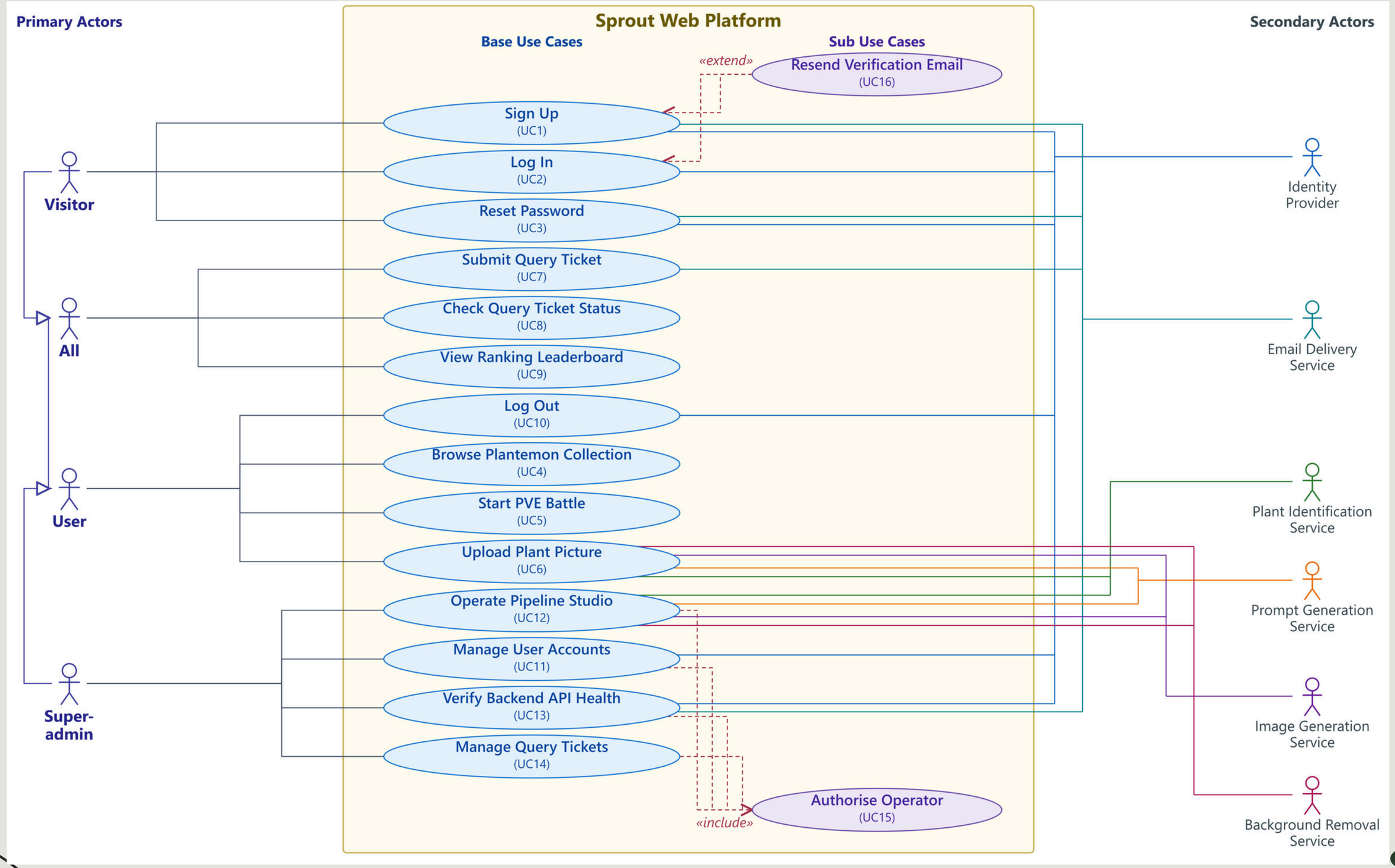
Primary (6 total)

- **All (anyone who reaches the platform)**
- **Visitor (No Session)**
- **User (Verified session)**
- **Superadmin (a User holding the operator grant)**

- **Misuser (Hostile intent)**
  - Every control refuses.
- **Careless User (Honest Mistakes)**
  - Code Handling/control mitigate risk and cost.

Secondary (6 total)

- **Identity Provider (Firebase Auth)**: creates identities, verifies session tokens.
- **Email Delivery (Resend):** verification links, reset codes, ticket mail.
- **GenAI Pipeline**
  - **Plant Identification (Plant.id)**
  - **Prompt Generation (NVIDIA or Gemini)**
  - **Image Generation (FLUX or Gemini)**
  - **Background Removal (withoutBG)**


Primary Actors
Sprout Web Platform
Base Use Cases
Sub Use Cases
Secondary Actors
«extend»
Resend Verification Email
(UC16)
Sign Up
(UC1)
Log In
(UC2)
Reset Password
(UC3)
Submit Query Ticket
(UC7)
Check Query Ticket Status
(UC8)
View Ranking Leaderboard
(UC9)
Log Out
(UC10)
Browse Plantemon Collection
(UC4)
Start PVE Battle
(UC5)
Upload Plant Picture
(UC6)
Operate Pipeline Studio
(UC12)
Manage User Accounts
(UC11)
Verify Backend API Health
(UC13)
Manage Query Tickets
(UC14)
Authorise Operator
(UC15)
«include»
Visitor
All
User
Super-
admin
Identity
Provider
Email Delivery
Service
Plant Identification
Service
Prompt Generation
Service
Image Generation
Service
Background Removal
Service

Misuse cases

Every control is an implemented alternative flow, except NM1, which is the main flow of UC6 at steps 9 and 10

Hostile; threatens a use case (9 total)

MC1 **Harvest credentials** by guessing · UC2 alt 3a, 3b
MC2 Enumerate registered accounts · UC3 alt 4a
MC3 **Guess** a reset one-time **password** · UC3 alt 11a, 12a, 13a
MC4 **Exhaust** the paid AI **pipeline** · UC6 alt 4a, 4b, 4c
MC5 Farm rewards by replaying actions · UC5 alt 9b, 15a
MC6 **Flood** the enquiry **queue** · UC7 alt 3a
MC7 Probe another submitter's ticket reference · UC8 alt 4a
MC8 Reach the console without the grant · UC15 alt 3a, 4a
MC9 Read another player's collection · UC4 alt 7a

Non-malicious; no hostile intent (7 total)

NM1 **Rescan** expecting a different result · UC6 steps 9, 10
NM2 **Abandon** a losing battle and **restart** · UC5 alt 15a, 16a
NM3 Resubmit an enquiry that seemed lost · UC7 alt 5a
NM4 Clean up the wrong data set · UC11 alt 5a, 5b
NM5 Test production, calls are real · UC13 alt 5b
NM6 Disclose the reset code to a helper · UC3 alt 12a, 13a
NM7 Never complete email verification · UC1 alt 8a, UC2 alt 6a

Functional Implementation



Frontend

+ Login/Signup/Reset Password **P0**
+ Landing page **P0**
+ Browsing Account's Avatars **P0**
+ Upload Pics **P0**
+ PVE game **P1**
+ Submit Query Ticket **P1**
+ Super Admin Dashboard **P2**

Backend

- Firebase Auth Integration **P0**
- Cross-platform Accounts Database **P0**
- Avatar / Plant Data API **P0**
- GenAI Image-to-Sprite Pipeline **P0**
- Query Ticket API **P1**
- Email Server API **P1**

Non-Functional Requirements



1) Budgets

- Prompt budget (Flux's 800 characters)
- Latency (110s deadline, not p95)

2) Evaluation and dex gate

- Programmatic evaluations and automated approval process
- Human-in-the loop

3) Cost scalability

- Sprites are saved globally
- Generation cost: O(species) not O(users)

4) Observability on Studio

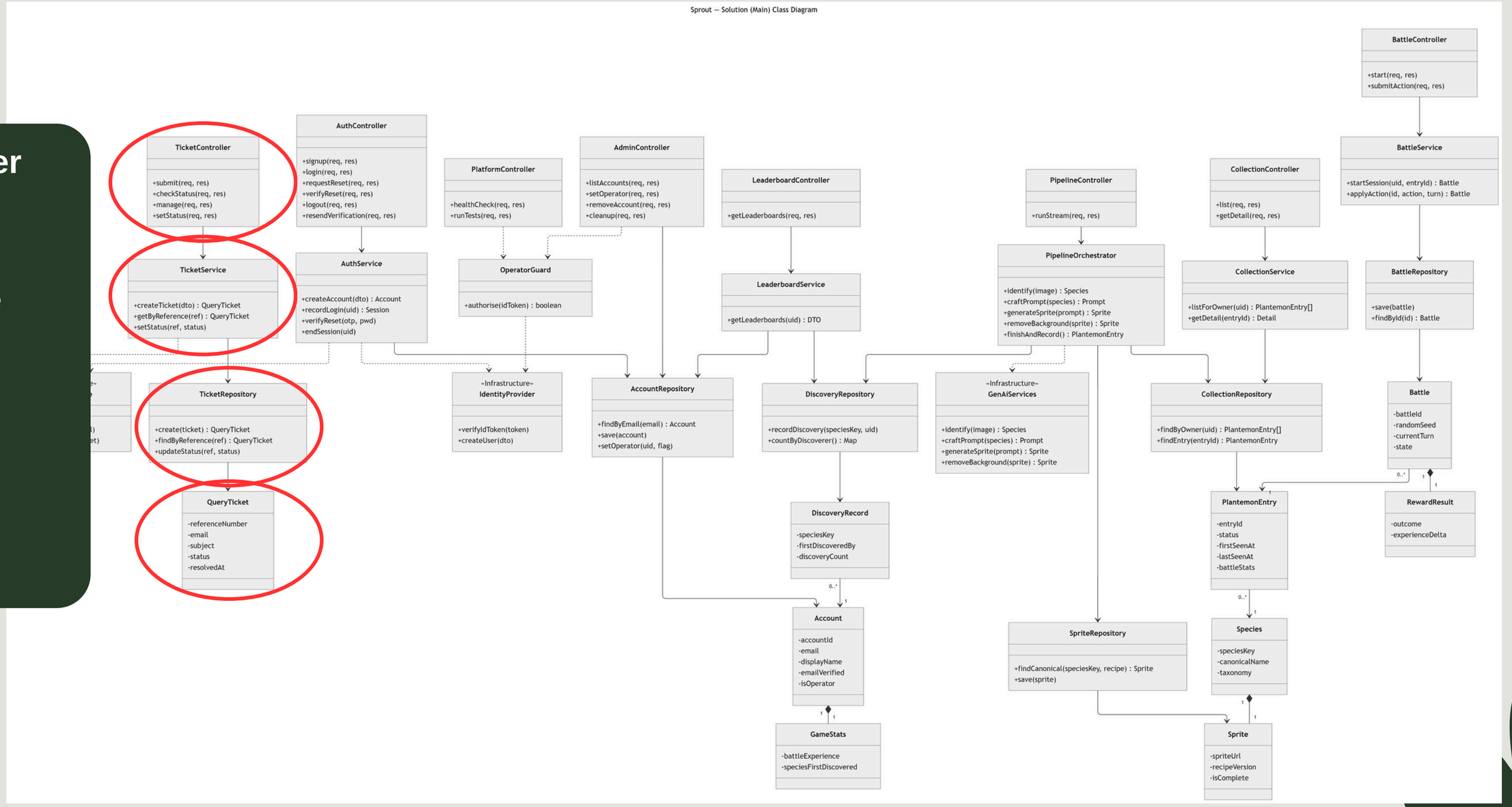
Class Diagram ✦

Controller

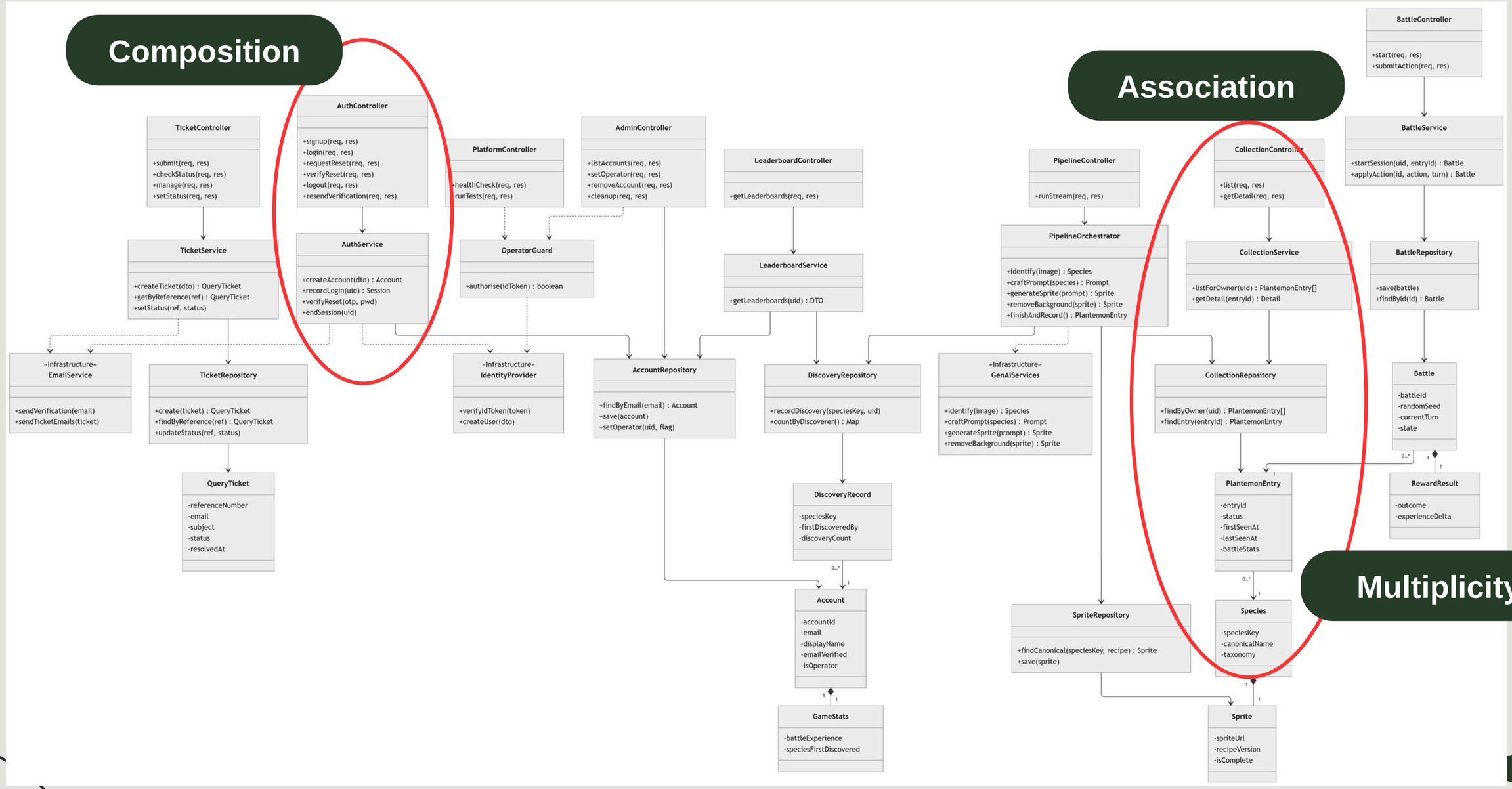
Service

Repo

Entity



Class Diagram ✦



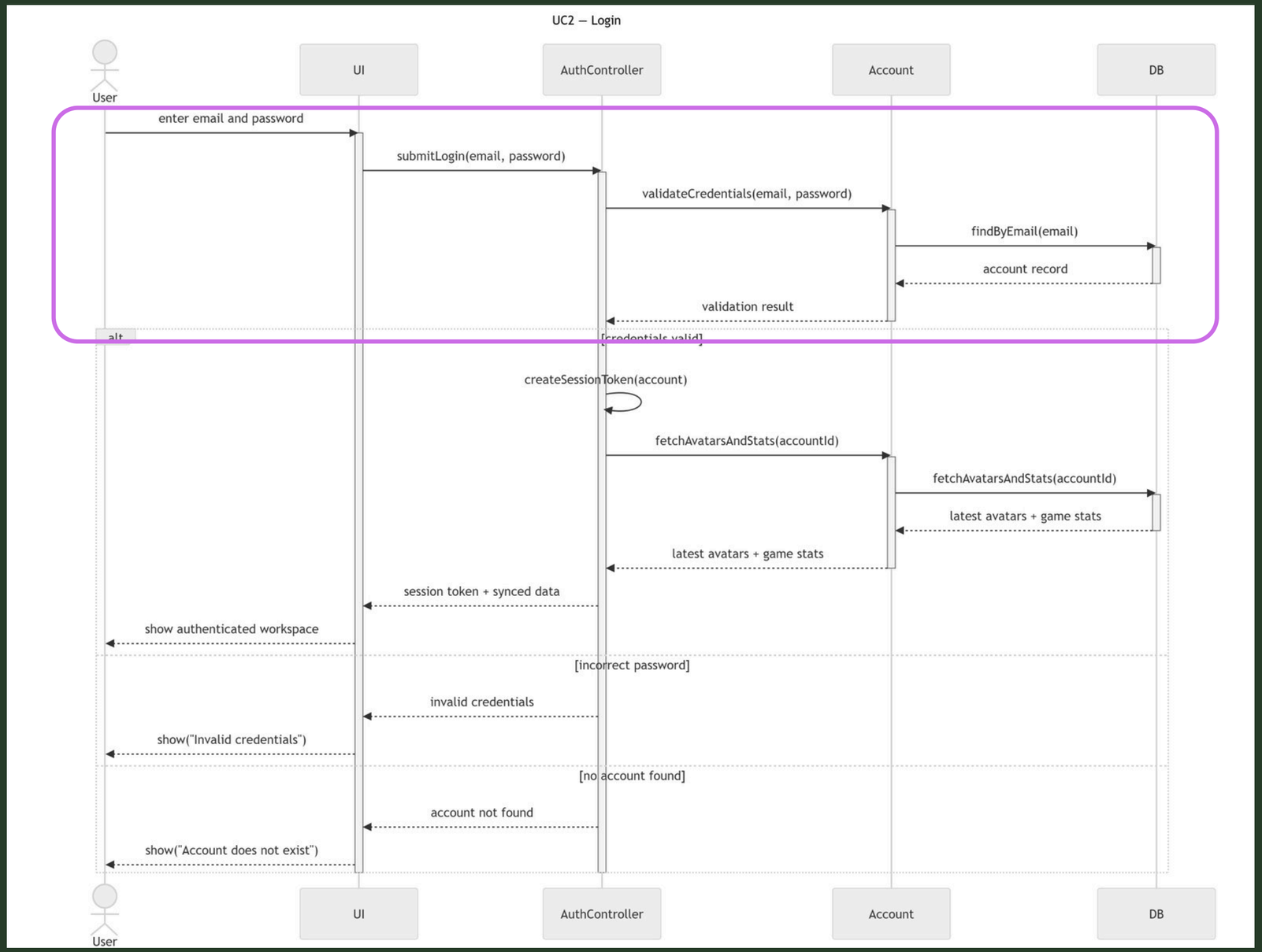
UC1 Signup



Verification Roundtrip

Validation alt fragments

UC2 Login

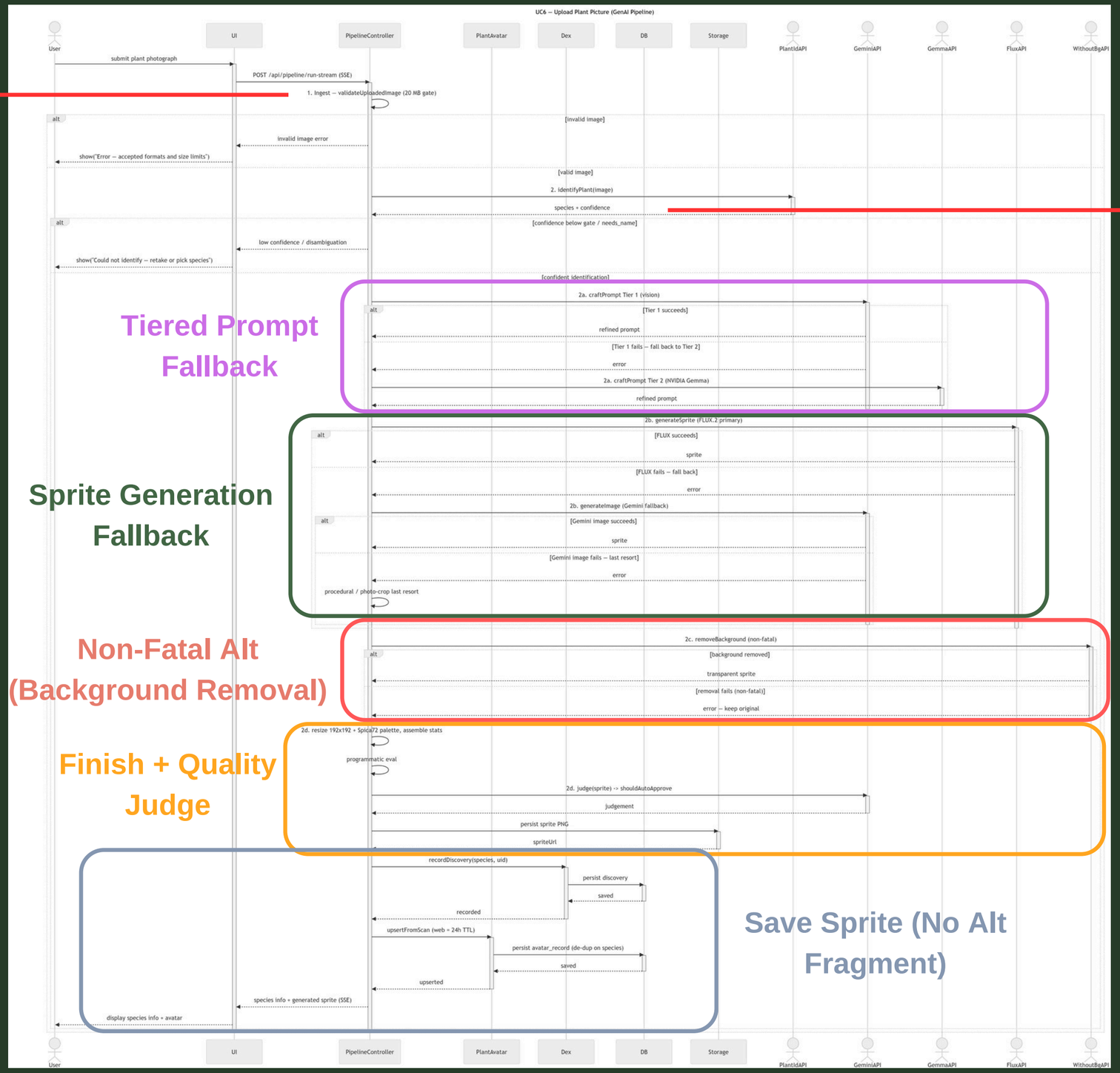


Client-Side Authentication

UC6 Upload Plant Picture

Uplaod Validation

Image Confidence Gate



Tiered Prompt Fallback

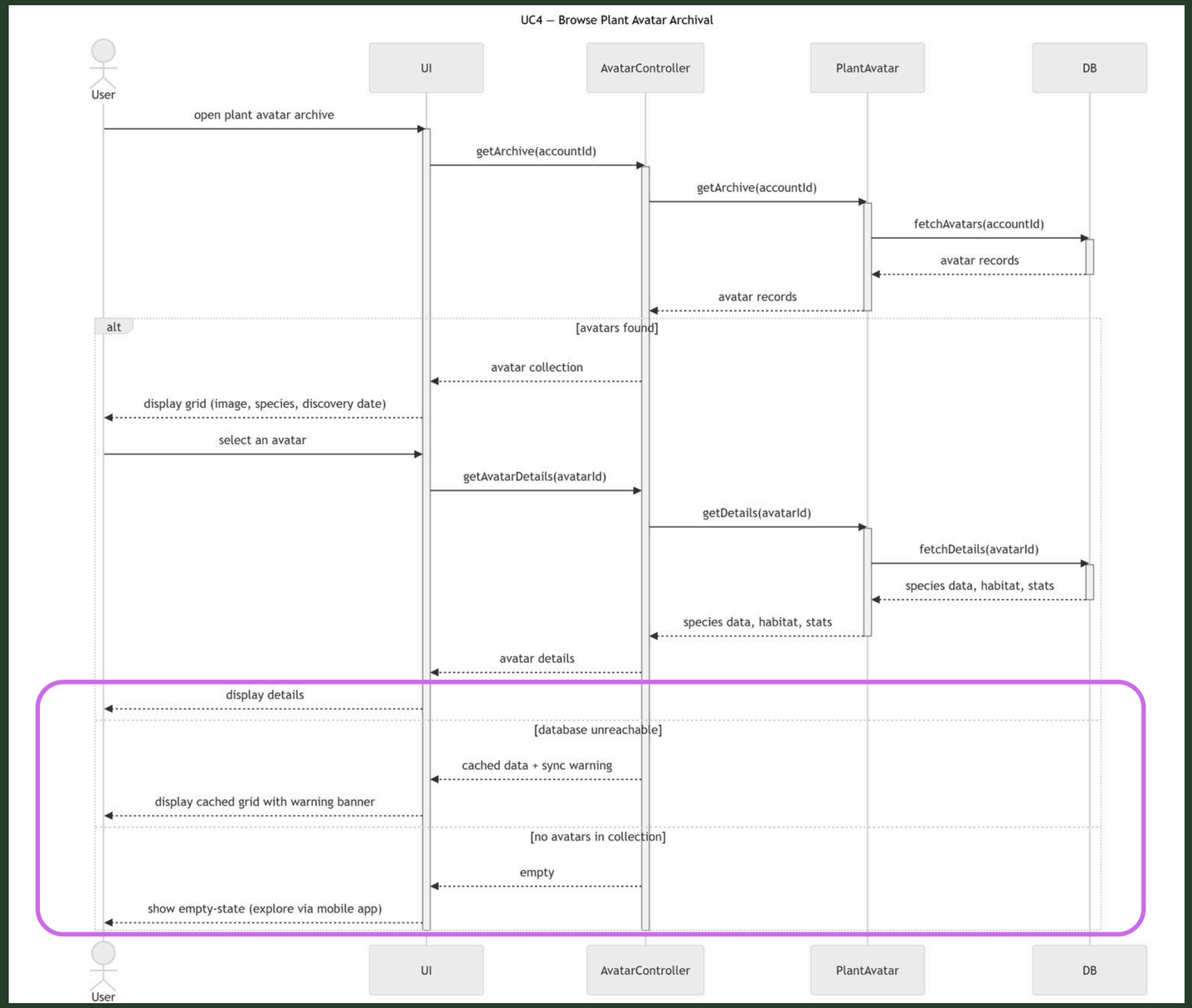
Sprite Generation Fallback

Non-Fatal Alt (Background Removal)

Finish + Quality Judge

Save Sprite (No Alt Fragment)

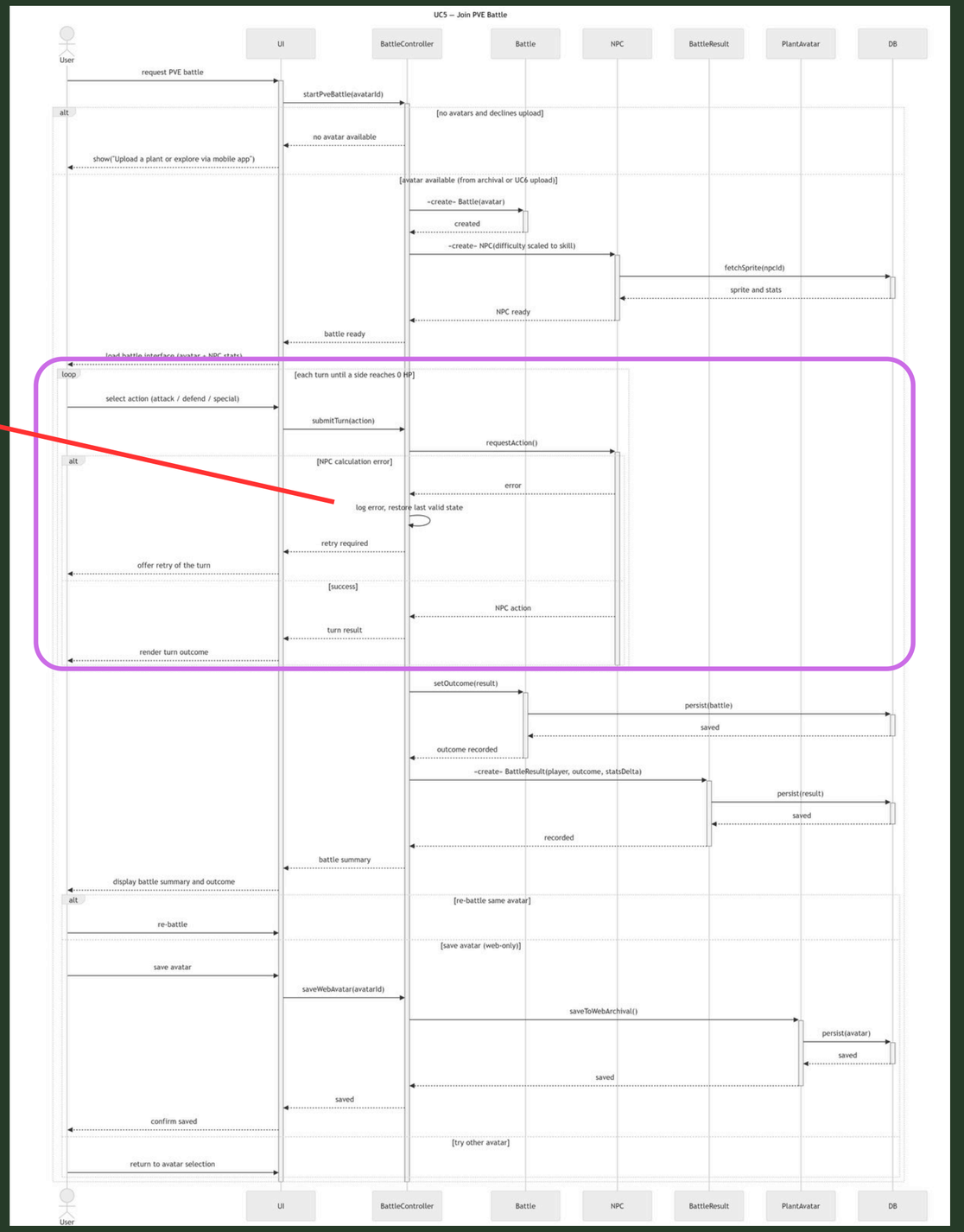
UC4 Archive



Cached Fallback

UC5
PVE Battle

Mid-turn Error Recovery



Loop until HP turns 0

Cloud Native Design & Resiliency

Stateless & Disposable Design Pattern	• **all states live in Firestore:** crash, restart, and redeploy are non-events
Liveness ≠ readiness	• **2 separate health checks (**alive vs serve requests**)** • **Separation means a sick dependency routes traffic away instead of triggering restart loops**
Resilient by Design	• **graceful shutdown -** deploys drain instead of dropping requests • **fail-closed auth -** missing config locks down rather than opens up • **per-route rate limiting -** one bad actor can’t take down the shared tier

CI-verified: 464 MB image · non-root · stops in 1 s on SIGTERM · live readiness 105 ms · 9 Aug 2026

Docker

Problem	• **Same dependency** (sharp, bcrypt) **compiled to different binaries on different operating systems**
What this meant	• **Passing on local machine ≠ Passing in production**
What we did	• **package the server as a Linux Docker image, runs in production on Render**

The exact **server image** we build and test is what runs in **production** on Render

Testing Objective

What we set out to prove

Every use case does what its diagram says

- The error paths too, not only the happy path

You cannot reach another user's data
An unverified account cannot reach gameplay

When an outside service fails, nothing breaks inside
No half-written data is left behind

Done means every CI job green: ten test groups, the E2E job, the container jobs. A red run blocks the merge.

Testing Strategy

Different bugs live in different layers		How
Backend	**Bottom up, call graph**	Database layer first, then the routes on top of it
Frontend	**Top down, from the user**	We click the page the way a user would
Battle Engine	**Property-based**	The computer invents the cases, we assert the rules

Test Levels and Scale

Level	What it proves	Files / cases
Unit + Property	Pure logic and engine rules	counted inside the two rows below
Integration (server)	Route to repository, on the real emulator	45 files / 616 incl. unit
Integration (client)	Page, router and context, network stubbed	28 files / 309 incl. unit
Robustness	Upload gate under mutation fuzzing	18 files / 155
End-to-end	Whole user journey in a real browser	13 journeys

1,093 test cases across 5 test levels

Input Domains & Data Types

IMAGE UPLOAD

Payload: 0 to 15,728,640 bytes, so 15 MB

Shortest edge: at least 16 pixels

Total pixels: at most 40,000,000

Format: jpeg, png or webp only

TEXT FIELDS

Password: 8 to 72 bytes, bcrypt cuts at 72

Display name: 1 to 50 characters

Ticket message: 1 to 2,000 characters

Inquiry type: fixed list, nothing else accepted

Boundary tests use these ranges: one value inside, one on the edge, one outside

Unit Test Cases

SAMPLE

5 of our 1,093 test cases. The report documents 90 of them in full.
Boundary values (based on domains of previous slide)

ID	Tier	Test case	Expected	Result
TC-UC5-10	Property	Engine rules under generated input	No negative damage, no health above max	**Pass**
TC-UC1-06	Unit	Display name at 50, then at 51	Accepted at 50, rejected at 51	**Pass**
TC-UC8-07	Unit	Ticket subject past its limit	Rejected, 400	**Pass**
TC-UC8-09	Unit	Inquiry type not on the list	Rejected, 400	**Pass**
TC-UC5-04	Unit	A move that is not in the moveset	Rejected, battle state unchanged	**Pass**

Integration testing



SERVER — call graph, bottom up

Routes → services → repositories.

Engine and Firestore repositories **proven first**, then the routes above them.
a **route failure points at the route**.

Repositories run against the Firestore Emulator, not mocks.

CLIENT — top down from the caller

Because React pages are **driven by user events**, not called from below

Pages rendered whole, with **real router and real child components**. Only the network boundary is mocked/stubbed.

Proves page ↔ router ↔ context integration



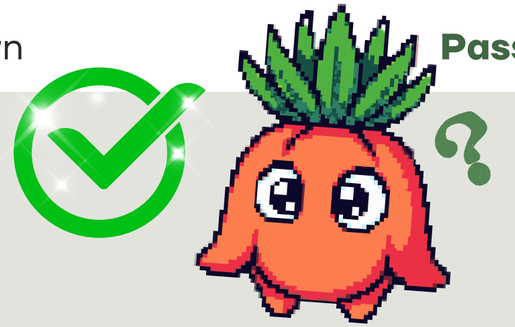




Integration test cases

Server rows run against the Firestore Emulator; client rows render with real router and context, network stubbed.

ID	Tier	Test case	Expected	Result
TC-UC4-03	Server	Archive holding another user's records	Only caller-owned avatars listed	**Pass**
TC-UC3-06	Server	Five concurrent wrong OTP attempts	One issuance invalidated, no double spend	**Pass**
TC-UC5-05	Server	Two requests for the same turn	One resolves, other gets stale snapshot	**Pass**
TC-UC8-10	Server	Email delivery fails (alt flow 5a)	Still 201, ticket still persisted	**Pass**
TC-UC4-08	Client	Sprite image fails to load	Falls back to CSS-rendered plant	**Pass**
TC-UC4-10	Client	Archive request fails	Retry control shown	**Pass**



Fuzzy testing (Image)



Image uploading during Upload Plant Picture, as part of sprite creation

Random baseline



Sends 10,000 payloads to gate

- Pure noise should be rejected.
- Rejection reason are provided (not_base64, unreadable).

Mutation-based fuzzing



Test validateUploadedImage() function

- bitflip
- truncate
- header_corrupt
- format_confusion
- extreme_resize
- pixel_noise
- exif_abuse
- not_an_image

Fuzzy testing assumes runs are free. They are not (for images)!!
A separate live runner was built (server/scripts/fuzz-pipeline-live.ts, npm run fuzz:live).

Fuzzy testing (Text)

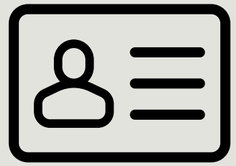
Required fields during account creation or ticket creation



Adversarial corpus testing with the same oracle

Attacking Joi scheme for UC8

- Injection payloads
- Unicode normalisation and zero-width abuse
- reference-number forgeries
- email grammar edge cases
- length boundaries
- ReDoS probes







System E2E testing



Set up:

Chromium → React → Express → Firestore, Auth and Storage

- No substitutions
- USE_MOCK_APIS

e2e suite covers:

- 13 user journeys
- 6 specs
- (CI) runs on every PR
- ~2 mins, zero secrets so works on any fork and can't silently skip

Exploratory Testing:

- Claude in Chrome
- 7 agents to comb through dead code
- 10 agents simulate personalities to break website



Playwright

Manual E2E Testing

Examples of Path Checked

- Landing -> Sign Up -> Play
- Landing -> Login -> Play
- Landing -> Login -> Reset Password -> Reset Successful -> Landing/Login
- Landing -> Contact Us -> Submit Successful -> Submit Another
- Landing -> Login -> Archive -> Battle
- Landing -> Login -> Scan -> Upload -> Drag & Drop -> Generate Sprite -> Battle/Generate Another/Archive/Download

Found and fixed (7 total)

- Duplicate identity: form then Google signup on one Gmail locked the user out
- Role-based navbar
- Ticket follow-up
- Content humanisation, standardisation and UX improvement (location, padding, hyperlinks)
- File validation was skipped during the upload process

Implementation challenges

Algorithmic **1 of 1**	**Battle stats** were not reproducible	now derived from the **plant species** itself
Engineering **1 of 8**	Render silently blocks outbound email through **SMTP**	we send over an **HTTPS** API instead
Engineering **2 of 8**	Verification email only reaches our own **admin inbox**	**needs a paid domain**, so we say so rather than claim it works
Testing **1 of 2**	E2E scan test was failing for test logins	both login paths now seen by the scan stream

11 challenges are documented in the report

Sustainability, Diversity & Inclusion



Sustainability	**Real local biodiversity awareness & education** (200 SG species) **One plantemon generated per species** (to keep AI's energy footprint low by limiting generation)
Diversity & Inclusion	**App is explorable without an account or mobile device** (accessible web app) **Plants are named the way locals refer to it** (Centella asiatica = Pegaga)

UNSDGs









how feedback was handled from PMs



After PM1 (Requirements)	After PM2 (Design)	After PM3 (Test)
Preconditions weakened to NIL in 13 of 16 use cases (handled in alternative flow) **Reset Password** is its own base use case Database, storage and battle engine dropped as **secondary actors** (Self-implemented code)	**MVC vs BCE standardised**: the view is the boundary **Email Service** no longer drawn talking to the user (has **adapter**) System **acknowledges before delivery**, so every **sync msg** shows a **complete round trip** **Alt. boxes stacked**, never nested. **Timeouts** modelled as explicit **alt. branches**	Every test case **declares data type and domain** (e.g. integer 0–100), so boundary values and equivalence classes are derivable **Fuzzing** given a valid/invalid taxonomy. **Frontend finetune**: Plantemon naming, persistent navbar, 4 Superadmin dashboards

Beyond the classroom learning

1) Understanding Non-Functional Requirements (Performance, Security)

2) Use Case Documention;

- Usage of Actor Generalisation for clearer representation of ‘Superusers’
- Documenting Misuse Cases



3) Test Suite; beyond black-box and white-box

- **Property**-based testing
- **Robustness Fuzzing** mutation-testing
- **Automated E2E** test (Playwright)



4) Deployment;

- **CI**: GitHub Actions, Firestore emulator
- **CloudNative**: Docker



Conclusion







Summary

- End to End Web Application
  - Frontend, Backend, Native Game Engine, Services (GenAI), Serverless Database, Vercel & Render Hosting (CI/CD)
- Comprehensive SDLC
  - 16 use cases → class, solution and sequence diagrams → 1,093 Passed tests across 95 files

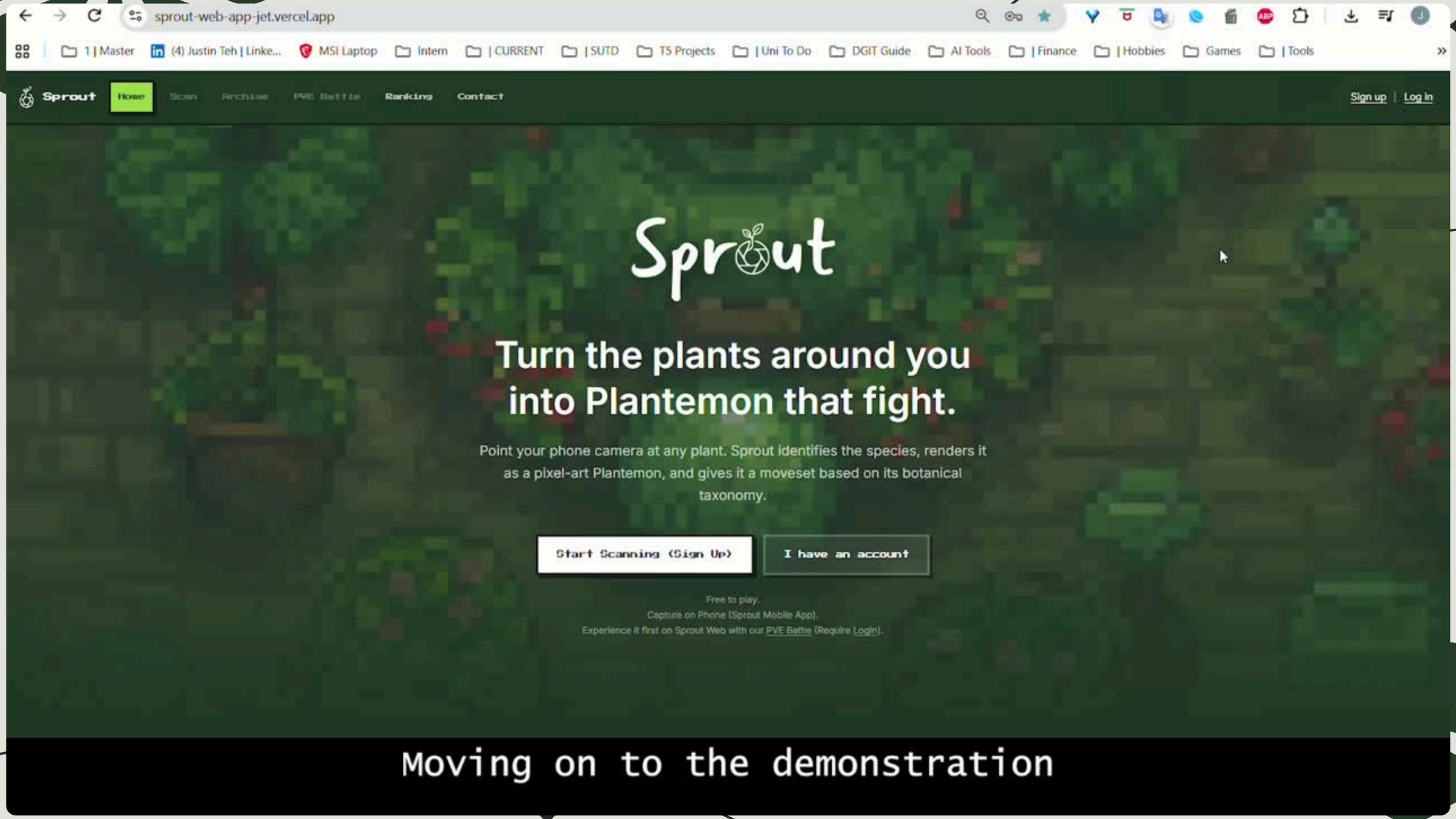
Takeaway

- Appreciation for Software Documentation & Spec-Driven Dev in SDLC
- Alternative Flow vs Misuse Cases (test vs mitigation)
- Importance of Manual E2E test (reveals what is not obvious in Use Case Alternative Flow)

Workload Distribution

Requirement Documentation (Use Case)	Justin, Andrina. Li Xiang
Design Documentation (Class, Solution, Sequence Diagrams)	Li Xiang, Omar
Checkoff, Video, Administrative	Andrina
Code Base	**Frontend**: Justin, Omar, Nat **GenAI Sprite Pipeline**: Nat **Backend, Database, Hosting, Frontend (PVE)**: Zhi Feng
Cloud Native Integration	Zhi Feng
Test Suite Implementation	**Unit, Integration, Robustness (Text & Image Fuzzing)**: Nat **Unit, Integration, Auto E2E (Playwright)**: Zhi Feng **Manual E2E**: Justin, Omar

Demo



Moving on to the demonstration

Q&A

By Cohort 3 Team 2

Scan. Grow. Battle.

Sprout

Web Platform & B2B Showcase for Nature-Based Gamification

Nathaniel Sim, Justin Teh, Omar, Andrina, Li Xiang, Zhi Feng